PART F – FINAL REGRESSION TESTING & COMPARISON VALIDATION

Test ID	Target Feature	Outcome Before Refactoring	Outcome After Refactoring	Engineering Improvement / Observation
TC-01	Basic Element Insert	Functional; uses global state.	Functional; encapsulated in DynamicArray.	Data is safely isolated inside a managed class instance.
TC-02	Dynamic Memory Resize	Memory leak when old heap buffer is replaced	Resize frees old buffer with delete[]	Prevents repeated heap leaks during capacity expansion.
TC-03	Traversal Output	Off-by-one heap over-read possible.	Clean traversal from 0 to size-1.	Bounds condition changed from <= to <.
TC-04	Linear Search – found	Returns index for a found target.	findItem() returns the target index using one pass.	Search remains O(n) worst case with cleaner logic.
TC-05	Linear Search – absent	Returns -1. .	Returns -1 when the target is absent.	Correct negative-result handling is retained.
TC-06	Element Deletion	Shift uses redundant nested-loop structure.	Single-pass O(n) shift.	Less unnecessary loop overhead and clearer implementation.
TC-07	Negative Index	Invalid input handling was incomplete.	index < 0 is explicitly rejected.	Defensive validation covers negative input.
TC-08	Upper Boundary Check	Invalid index can access outside valid data.	index >= size is rejected safely.	Prevents invalid array access and possible memory corruption.

TC-09	3×3 Matrix Transpose	Transpose works for fixed 3×3 input	Same 3×3 transpose logic retained.	Matrix behavior is preserved; dimensions remain fixed in this version.
TC-10	Program Cleanup	Dynamic memory remains allocated on exit	Destructor releases data automatically	Heap ownership is handled through object lifetime